

ИССЛЕДОВАНИЕ

1 АПРЕЛЯ 2026 ГОДА

На пути к бесконечным контекстным окнам: нейронное сжатие KV-кэша



Чарльз О'Нил (Базеттен)



Алекс Сандомирский (Baseten)



Гарри Партридж (Базеттен)

TL;DR

Мы сжимаем KV-кэш больших языковых моделей в 8 раз за миллисекунды, сохраняя более 85 % фактической точности в разных областях. Вот как это работает.

Введение

У современных больших языковых моделей есть проблема с памятью. И речь не о том, что ее можно решить за счет увеличения объема контекста, а о том, что в конце диалога агент забывает все, что знал. Современные подходы к сохранению данных в памяти несовершенны. Либо они требуют больших затрат и не сохраняют данные без потерь (полный кэш KV, линейно масштабируемый в зависимости от контекста), либо сильно сжимают данные и допускают потери (тонкая настройка, файлы памяти в стиле Markdown, RAG). Третьего не дано.

Люди устроены иначе. В рабочей памяти без потерь мы удерживаем примерно семь элементов (магическое число Миллера), что намного меньше, чем контекстное окно больших языковых моделей. Но у нас есть большая структурированная кратковременная память, которая не является ни дословной, ни полностью сжатой копией долговременной памяти. Этот промежуточный уровень — рабочая память с

потерями, но высокой точностью — позволяет нам проводить совещания, читать документы или работать над многодневным проектом, не забывая ничего и не запоминая каждое слово.

Мы считаем, что создание этого промежуточного уровня памяти является необходимым условием для непрерывного обучения больших языковых моделей, то есть такого обучения, которое не ограничивается онлайн-обновлениями LoRA по мере поступления производственных данных. Для создания большой языковой модели, которая накапливает знания в ходе диалогов, необходимы три возможности:

(1) сжатая рабочая память, расположенная между кэш-памятью KV без потерь и долговременной памятью с потерями, хранящейся в весовых коэффициентах

(2) способ, с помощью которого модели смогут эффективно работать с этой памятью в цикле и, возможно, даже научиться самостоятельно управлять этой памятью (решая, когда сжимать данные, что сохранять, а что удалять).

(3) механизм, позволяющий проецировать то, что было усвоено в сжатой памяти, обратно на нейроны, превращая опыт в устойчивые знания (как это происходит у людей во время сна)

В этой статье рассказывается о наших успехах в реализации первой возможности. В крайних случаях, когда память больших языковых моделей (кэш KV) не используется, каждый токен сохраняется дословно, но в некоторых моделях с открытым исходным кодом контекст из 128 тысяч токенов занимает сотни гигабайт, и на каждом этапе декодирования обрабатывается весь этот объем. Последние методы сжатия показали, что этот кэш можно заменить на гораздо более компактный, сохранив при этом поведение модели. [Attention Matching](#) (Цвайгер и др., 2026) разбивает задачу на части для каждой KV-головки, выбирает компактные ключи и аналитически вычисляет смещения и значения для создания компактного кэша. Аналитическое решение зависит от контекста. [Cartridges](#) (Эйюбоглу и др., 2025) оптимизирует компактные кэши с помощью градиентного спуска. Оба метода обеспечивают высокое качество при высокой степени сжатия, но оба выполняют контекстную оптимизацию во время логического вывода, тратя от нескольких секунд до нескольких минут или даже часов на сжатие каждого нового контекста. Для слоя рабочей памяти нам нужно что-то, что работает со скоростью прямого прохода.

Изначально эта работа была вдохновлена идеей разреженных автокодировщиков (sparse autoencoders, SAE). В классическом разреженном кодировании для каждого входного сигнала выполняется итеративная оптимизация, чтобы найти разреженное представление в фиксированном словаре. В разреженных автокодировщиках этот процесс автоматизирован: они обучают один кодер, который генерирует разреженные коды за один прямой проход, жертвуя качеством кодирования для

каждого входного сигнала ради ускорения на несколько порядков. Attention Matching и Cartridges выполняют эквивалент разреженного кодирования, запуская индивидуальную оптимизацию для каждого нового контекста.

Вопрос, с которого начался этот проект, заключался в следующем: можем ли мы сделать то же, что и самообучающиеся нейронные сети для обучения словарям, но для сжатия KV-кэша? Научиться использовать фиксированный кодер, который преобразует любой полный кэш в компактный за один прямой проход, обобщая данные в зависимости от контекста так же, как самообучающиеся нейронные сети обобщают входные данные?

STILL — это узкое место в восприятии, которое выполняет именно эту функцию.

Фиксированный набор изученных векторов запросов взаимодействует с полным KV-кэшем, формируя компактные ключи и значения за один прямой проход. Это быстрая, дифференцируемая сжатая память, с которой большая языковая модель может взаимодействовать так, как если бы это был реальный контекст. При итеративном сжатии одного фрагмента, добавлении его к следующему, повторном сжатии и т. д. модель в принципе может обрабатывать документы произвольной длины, сохраняя память фиксированного размера. Воспринимающая модель обучается с помощью дистилляции по Кульбаку — Лейблеру: «ученик» (LLM + компактный кэш) должен соответствовать «учителю» (LLM + полный кэш) по распределению токенов-ответов. Вся модель LLM остается неизменной, градиенты получают только параметры воспринимающей модели (~7 млн на слой).

Идея хорошая, но для ее реализации потребовалось решить несколько неочевидных проблем. В этой статье мы описываем архитектуру, исправления, которые позволили добиться сходимости обучения, а также первые результаты, демонстрирующие эффективность подхода при разных коэффициентах сжатия, областях применения и масштабах моделей.

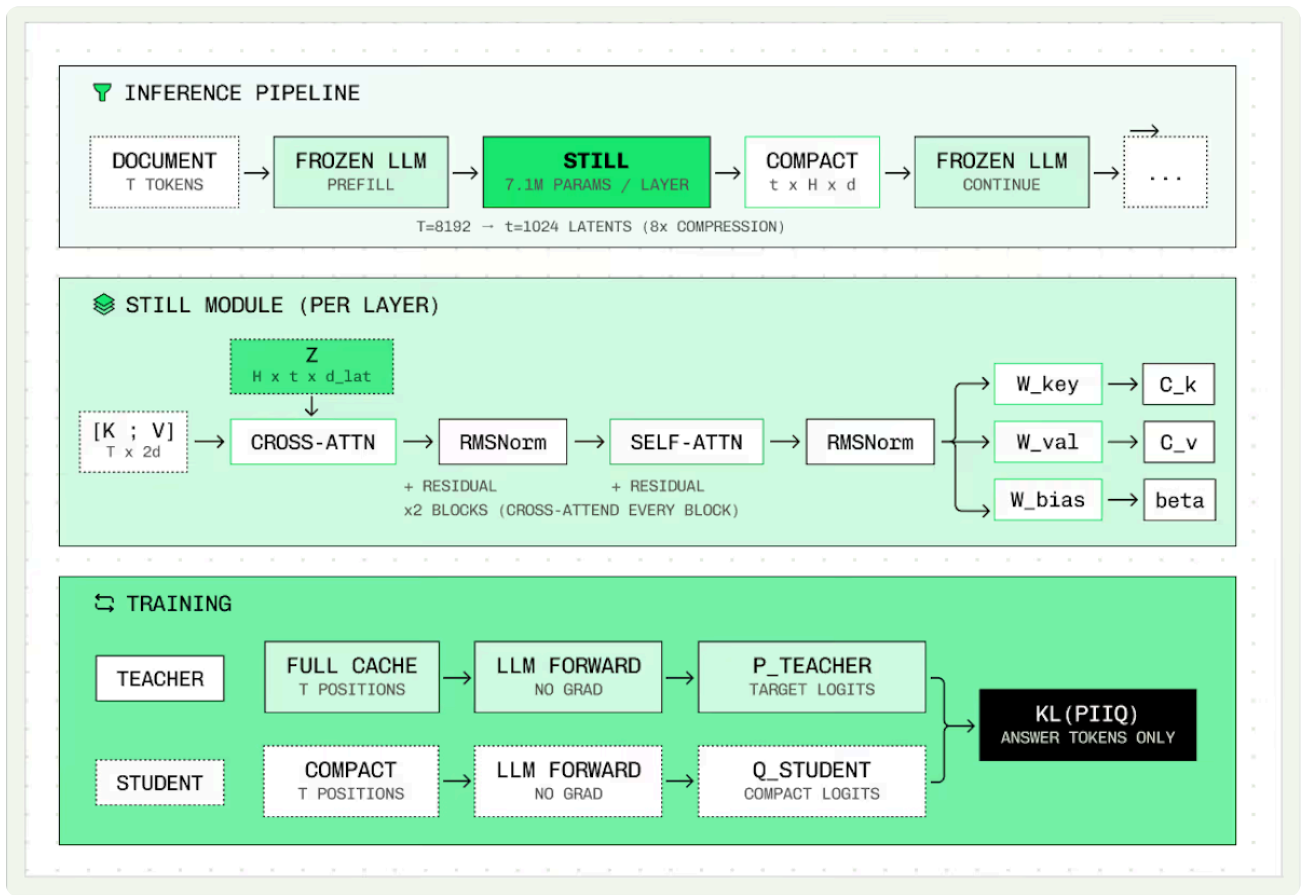


Figure 1: STILL compresses the KV cache at every layer of a frozen LLM. Learned latent queries cross-attend into the full cache, self-attend to coordinate, then project to compact keys, values, and attention biases. Trained end-to-end via KL distillation.

Method

Two recent methods have demonstrated that KV caches can be compressed with surprisingly little quality loss. Attention Matching (Zweiger et al., 2026) works analytically: for each KV head, it selects or constructs compact keys, solves for attention biases via NNLS, and fits compact values via least squares. The quality is strong, but the method requires reference queries -- and the queries that work well come from a "self-study" phase where the model generates continuations of its own context, adding minutes of per-context compute. Cartridges (Eyuboglu et al., 2025) takes an end-to-end approach: it gradient-optimizes a compact cache directly against downstream loss by backpropagating through the frozen LLM, essentially doing at inference time, for every new context, what we do once at training time. This produces excellent compact caches, but at a cost of minutes to hours per context.

Both methods deliver quality that validates the premise: the information in a KV cache can be preserved at high compression ratios. But both are too slow for a working memory layer. STILL compresses an 8K context in milliseconds; a single forward pass through a small perceiver, with no query generation, no self-study, and no per-context optimization. We also anticipate that as we move from MCQ-based training to training on repeated cache / model continuations (which we've already started on; see below), we will remove the

remaining self-study requirement from the training pipeline as well, making the entire system self-supervised. Direct quality comparisons to both methods are in progress and will follow.

Architecture

STILL operates independently at each layer of the frozen LLM. At layer l , the compactor takes the full KV cache (K_l, V_l) and produces a compact cache (C_k, C_v, β) , where T is the original sequence length, $t \ll T$ is the number of latents, H is the number of KV heads, and d is the head dimension.

The architecture is a standard perceiver:

1. *Latent queries.* A learnable parameter Z provides t query vectors per KV head group. These are the "questions" the perceiver learns to ask of any input cache.
2. *Cross-attention.* Latent queries attend into the concatenated key-value input $[K; V]$. This maps $(T, 2d)$ to (t, d_{lat}) in $O(T * t * d)$ time, linear in context length.
3. *Self-attention.* Latents attend to each other, enabling coordination, e.g., "I captured the medication dosage, you captured the patient name, we don't need to duplicate."
4. *Output heads.* Separate linear projections produce compact keys C_k , compact values C_v , and scalar biases beta from the refined latent representations.

The bias term beta corrects for attention mass lost during compression. In our earlier architecture (random init, 128 latents), we used a mass-matched beta offset of $\log(T/t)$ to force the LLM to attend to the compact cache; without it, the perceiver learned to produce caches the LLM simply ignored, achieving high utilization scores while fabricating content. With identity initialization, mass-matched beta is no longer needed: the compact cache starts as a near-copy of the input KVs, so the LLM attends to it from the first training step. Beta is instead learned end-to-end via a linear projection, zero-initialized.

Training

Each training step:

1. Prefill (no grad): Forward document tokens through the frozen LLM to obtain the full KV cache at all layers.
2. Teacher logits (no grad): Forward answer tokens with the full cache to obtain target distributions.

3. Compact (with grad): Run STILL at each layer to compress the full cache to t positions.
4. Student logits (with grad): Forward the same answer tokens with the compact cache.
5. Loss: $\text{KL}(P_{\text{teacher}} \parallel Q_{\text{student}})$ on answer tokens only.

Training data consists of extractive multiple-choice questions (MCQs) generated from long documents. Each MCQ tests whether a specific fact from the document is preserved in the compact cache. We use a two-stage generation pipeline: Claude Sonnet 4.6 generates questions, then generates distractors separately without seeing the source text (producing harder wrong answers).

We found on-policy data to be important. That is, answer text must be generated by the same model being compacted. Cross-model answers (e.g., Claude-generated answers for a Qwen compactor) sometimes produce catastrophic failure, i.e., the perceiver learns to shift the LLM's distribution toward the other model's patterns rather than preserving information.

Three fixes we made

The perceiver architecture is simple, but three non-obvious problems initially prevented training from working at scale.

RoPE-aware position encoding

Rotary position embeddings (RoPE) are applied to keys before they enter the KV cache. When the perceiver synthesizes compact keys as weighted combinations of original keys, it blends vectors that carry different positional encodings. A blend of RoPE-rotated keys from positions 5, 42, and 300 is not a valid RoPE-rotated key at any position; the result is a scrambled vector with no coherent positional meaning.

We solve this with a three-step pipeline:

1. *Un-rotate*. Before the perceiver, we apply inverse RoPE to strip positional encoding from the cached keys, recovering position-free representations.
2. *Compress with position-aware cross-attention*. The perceiver has its own internal RoPE on its cross-attention. Key positions match the original token positions $[0, T]$. Latent query positions are spread evenly across the same range via `linspace(0, T-1, num_latents)`, i.e., the positions scale by T/t , so each latent “covers” a proportional span of the original context. This gives the perceiver position awareness without entangling it with the LLM's RoPE encoding.

3. *Re-rotate*. After compaction, we apply forward RoPE to the compact keys at their evenly-spaced positions. Each compact key now carries a clean, well-defined positional encoding that the LLM's attention mechanism can interpret correctly.

Without this pipeline, the perceiver either operates on scrambled positional encodings (producing compact keys that confuse the LLM's attention) or treats all input positions as interchangeable (losing sequential structure). An early MoE training run spent 235 steps with completely broken positional encoding before the bug was identified (util = -1.83, indicating the compact cache was actively worse than no context).

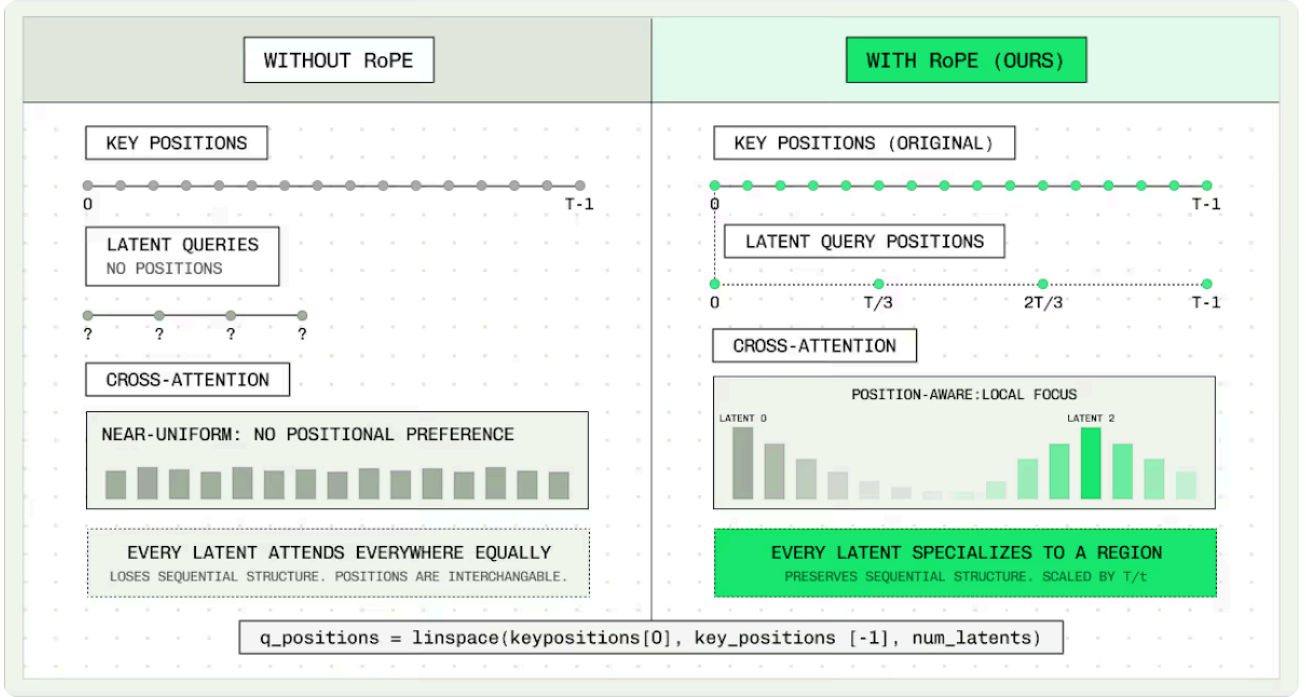


Figure 2: Step 2 of the un-rotate/compress/re-rotate pipeline. After stripping the LLM's RoPE (step 1), the perceiver's own cross-attention uses RoPE to encode where each input KV entry came from. Key positions match original token positions $[0, T]$. Latent query positions are spread evenly across the same range, scaled by T/t . After compaction, compact keys are re-rotated at their evenly-spaced positions (step 3).

Removing the final normalization

Standard perceiver architectures apply a final RMSNorm to the latent representations before the output heads. In our setting, this normalization constrains the compact key and value vectors to lie on a hypersphere of fixed radius in the latent space, regardless of the content. Real KV cache entries from the LLM have varying norms that carry information – layer norm in the transformer produces outputs whose norms reflect the magnitude of the residual stream updates. By forcing all latent vectors to have unit RMS before projection, the final norm discards this signal and forces the output heads to compensate.

Removing the final norm allows the perceiver to produce compact keys and values with the natural norm variation that the LLM's attention mechanism expects, improving convergence and final quality.

Identity initialization with attention biases

The original perceiver was limited to ~128 latents regardless of context length. At 256 latents, training matched but never exceeded 128-latent performance; at 512+, training diverged entirely. We attributed this to the LLM's attention capacity for synthetic positions.

The breakthrough came from initializing the entire perceiver pipeline as a near-identity function; at init, each latent should simply copy the input KV entry at its corresponding position. This requires coordinated initialization across the full pipeline:

- *Value pathway (identity chain).* $\mathbf{v_proj}$ and $\mathbf{out_proj}$ are set to identity matrices, and the output heads $\mathbf{W_key}/\mathbf{W_value}$ are set to identity patterns that extract the key and value halves of the latent representation, respectively. At init, the value pathway is a straight pass-through: whatever the cross-attention selects flows unchanged to the output KVs. At 1:1 compression this is an exact copy; at higher ratios, each latent produces a weighted average of its $\sim T/t$ nearest positions.
- *Q/K pathway (biased projections).* With the value pathway as identity, the remaining question is which input positions each latent attends to. We add bias terms to $\mathbf{q_proj}$ and $\mathbf{k_proj}$ in the cross-attention. The $\mathbf{k_proj}$ bias is set to a large constant vector ($\mathbf{q_hat} * 10$), making all projected keys content-independent at init – they all point in the same direction regardless of input content. The $\mathbf{q_proj}$ bias provides a matching query direction. With content stripped from the Q/K pathway, the only remaining signal that differentiates attention across positions is RoPE → the perceiver's internal positional encoding becomes the dominant factor in attention routing.
- *Zero-init residuals.* Self-attention output projections and later blocks' cross-attention output projections are zeroed, so only the first block's identity pathway is active at init.

The effect is that at initialization, latent i attends most strongly to the input positions closest to its RoPE position (position $i * T/t$), and copies those positions' keys and values through the identity value pathway. Training then refines this from a position-copying baseline toward content-aware compression. The identity init gives every latent a meaningful starting point and prevents the collapse where hundreds of latents converge to similar representations under a weak gradient signal. It should be noted that in this setup, $d_{lat} = 2d$

With this initialization, STILL scales monotonically from 128 to 8192 latents, with MCQ accuracy increasing from ~60% to ~95% as compression ratio decreases (Figure 1). The previous 128-latent ceiling was not a fundamental limit of LLM attention over synthetic positions – it was an optimization failure (although the architecture did need to change as well; see below).

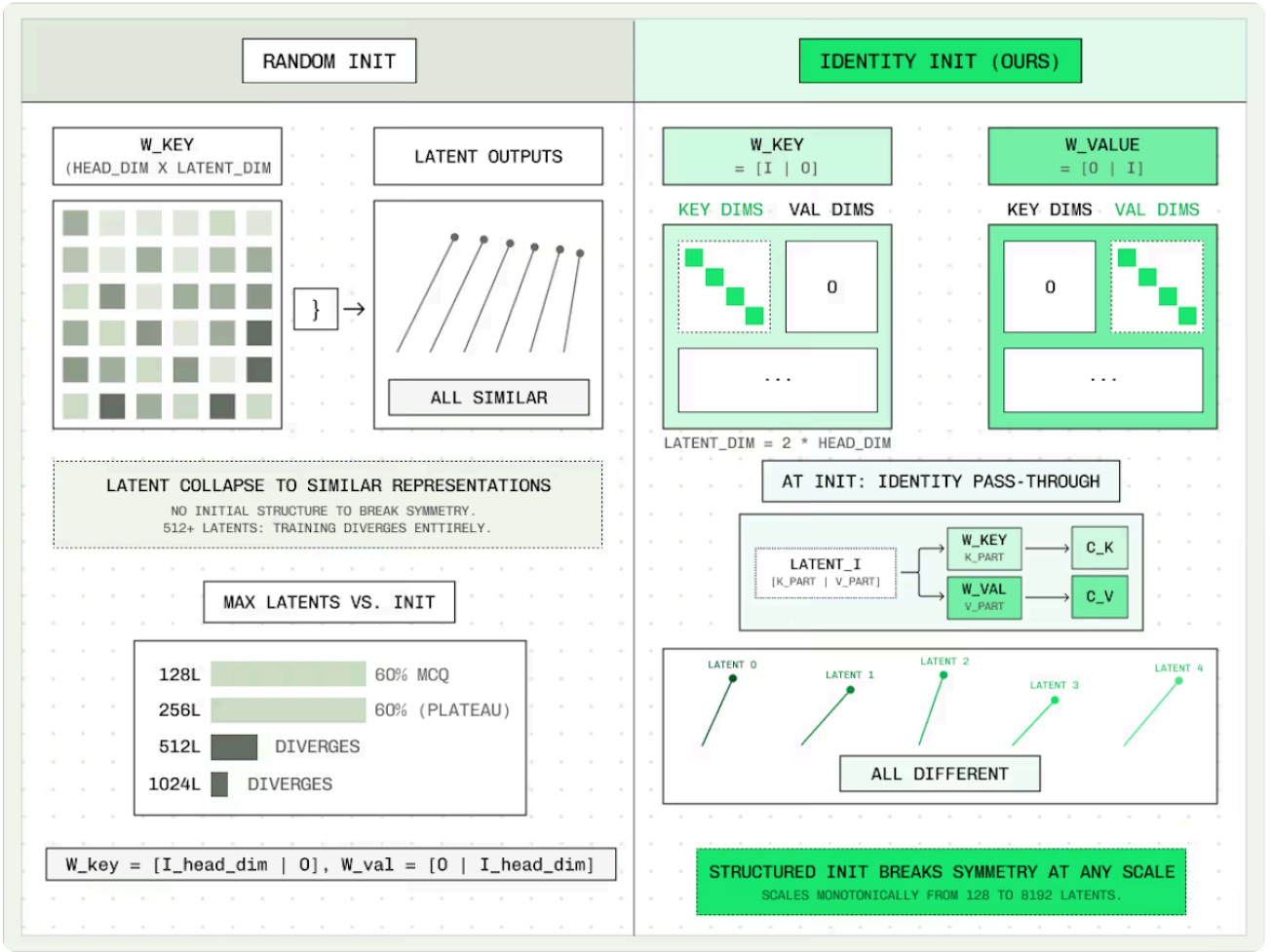


Figure 3: At init, the perceiver is a near-identity pass-through. The value pathway (v_{proj} , out_{proj} , W_{key} , W_{value}) forms an identity chain. Biased Q/K projections make all keys content-independent, so the perceiver’s internal RoPE is the dominant attention signal, i.e., each latent copies its positionally-nearest input. Training refines this into learned compression.

Results

All experiments use Qwen3-4B with 8192-token contexts unless otherwise noted. Training runs on 8x H200 GPUs with DDP. The perceiver has 2 blocks, 1 cross-attention head, 1 self-attention head, and latent dimension 256. Total trainable parameters: ~7.1M (0.18% of the base model). We evaluate on held-out extractive MCQs and continuation cross-entropy.

We report two main evaluation metrics. MCQ accuracy measures factual retention: can the model answer extractive multiple-choice questions about the document using only the compact cache? This is deliberately adversarial because each question targets a specific fact (a medication dosage, a date, a variable name) that the compactor must have preserved among thousands of tokens. It tests pointwise retrieval, the hardest task for lossy compression.

Utilization measures how much of the gap between no-context and full-context performance the compact cache closes, defined as: $utilization = (no_context_metric - compact_metric) / (no_context_metric - full_cache_metric)$.

A utilization of 1.0 means lossless compression (compact matches full cache). A utilization of 0.0 means the compact cache is no better than having no context at all. We compute utilization over cross-entropy (CE) and MCQ accuracy.

Scaling with latent count

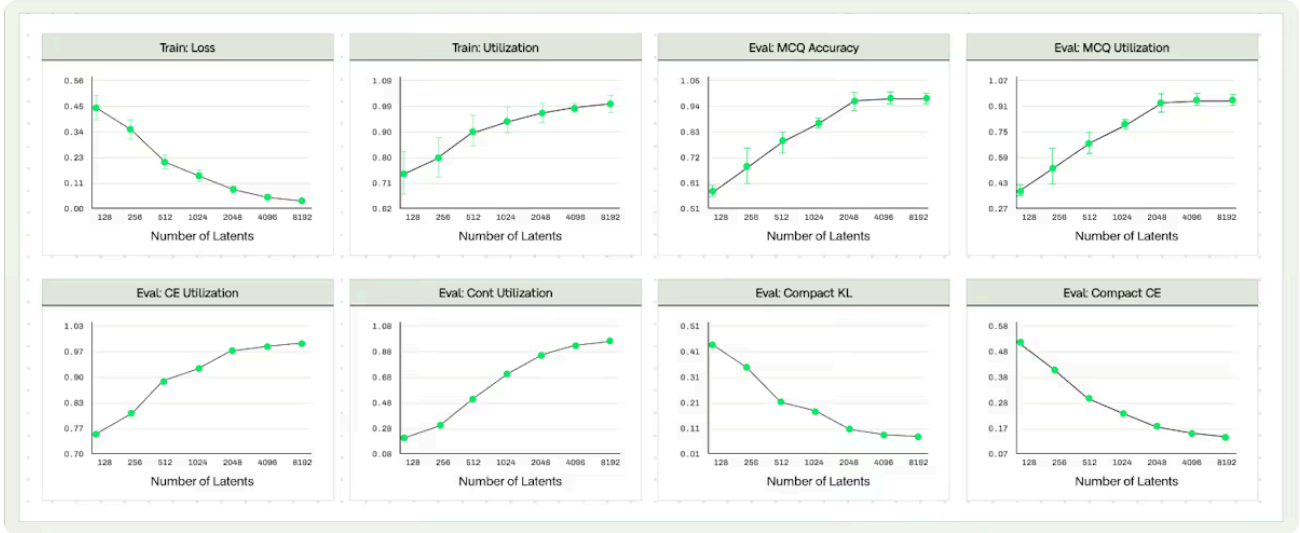


Figure 4: Metrics vs. number of latents (128-8192). Train loss, train utilization, compact MCQ accuracy, MCQ utilization, CE utilization, continuation utilization, compact KL, compact CE. Mean +/- std over tail steps.

Latent sweep with block-diagonal initialization. All metrics improve monotonically as the number of latents increases from 128 (64x compression) to 8192 (1:1, no compression). At 1024 latents (8x compression): MCQ accuracy ~85%, KL ~0.15, CE utilization ~0.93. The KL and CE are computed over the full context LLM's reasoning trace tokens in response to the MCQ prompt. Continuation here refers to the task of completing the next 128 tokens in the compacted passage when prompted with 64 cue tokens.

With identity initialization, STILL scales monotonically across the full range of latent counts. At 1024 latents (8x compression of an 8192-token context), MCQ accuracy reaches ~85% with utilization above 0.90. Even at 128 latents (64x compression), the perceiver preserves enough information for ~60% MCQ accuracy, which is well above the 20-22% no-context baseline.

The smooth scaling curve has a practical implication in that the compression ratio is a continuous knob, not a binary works/doesn't-work decision. Deployments can choose their operating point on the accuracy-memory tradeoff based on their latency and memory budget.



Figure 5: KL divergence training loss over 2900 steps for each latent count. 8xH200, DDP, financial MCQ dataset.

Cross-domain generalization

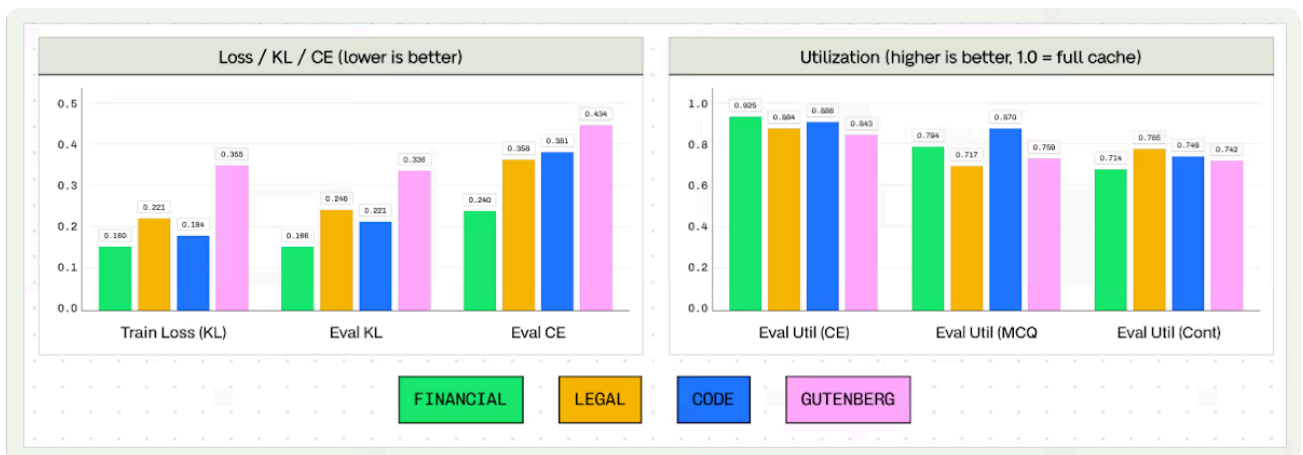


Figure 6: 1024L Block Diagonal. Loss/KL/CE and utilization by domain (Financial, Legal, Code, Gutenberg).

Domain-specific training at 1024 latents (8x compression). We trained separate STILL instances on four domains (financial filings, legal documents, source code, and Project Gutenberg literature) each with domain-specific MCQ datasets. The perceiver achieves strong compaction across all domains at 8x compression (1024 latents / 8192 context). The final MCQ accuracy is as follows:

Domain	MCQ Accuracy	MCQ Utilization
FINANCIAL	86.0%	79.4%
LEGAL	75.6%	71.7%
CODE	89.6%	87.0%
GUTENBERG	79.2%	75.9%

Financial and code documents compress most effectively (86-90% MCQ accuracy), likely because they contain structured, locally-concentrated information that maps well to individual latents. Legal and Gutenberg text are harder; they have more distributed, context-dependent information that is inherently harder to compress into discrete positions.

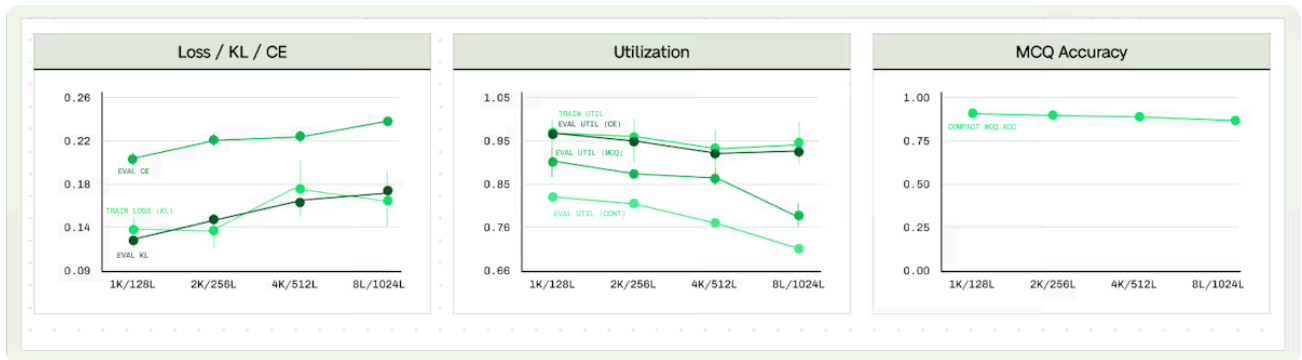


Figure 7: Constant 8x compression ratio. Varying context length (1K-8K) and latent count (128-1024). Financial MCQs. Identity init. Tail mean $\pm$ std.

Constant 8x compression with varying context length. Holding compression ratio fixed at 8x, we vary context length (and correspondingly the latent count). MCQ accuracy is stable at 85-92% across configurations, confirming that the perceiver learns compression as a ratio-dependent operation, not a latent-count-dependent one.

An important control experiment was holding the compression ratio fixed at 8x while varying the absolute context length and latent count. MCQ accuracy remains stable at 85-92% across all configurations (128 latents for 1k context through 1024 latents for 8k context), confirming that STILL learns compression as a function of the ratio, not the absolute number of positions (although there is a slight drop-off). This is a prerequisite for the variable-length generalization we discuss below.

Cross-domain transfer

MCQ COMPACT ACCURACY (%)

EVAL / TRAIN	FINANCIAL	LEGAL	CODE	GUTENBERG
FINANCIAL	87%	86%	77%	78%
LEGAL	74%	81%	87%	82%
CODE	70%	74%	77%	72%
GUTENBERG	72%	76%	76%	89%

CE UTILIZATION (%)

EVAL / TRAIN	FINANCIAL	LEGAL	CODE	GUTENBERG
FINANCIAL	68.4%	78.1%	76.7%	64.8%
LEGAL	59.4%	77.6%	73%	57%
CODE	52.7%	69.6%	74%	53.1%
GUTENBERG	65.5%	78.1%	78.9%	74.3%

Figure 8: 8K context, 1024 latents (8x compression), block-diagonal, 100 docs. Four heatmaps: MCQ Compact Accuracy (%), MCQ Utilization (%), Continuation CE (compact), CE Utilization (%). Rows = eval domain, columns = trained checkpoint.

Cross-domain transfer. Each cell shows performance when a perceiver trained on one domain (column) is evaluated on another (row). Diagonal entries (in-domain) are strongest, but off-diagonal transfer is surprisingly robust: 70-86% MCQ accuracy across all train/eval pairs. A perceiver trained on financial documents achieves 87% MCQ accuracy on financial eval, but also 74% on legal, 70% on Gutenberg, and 72% on code. The Code-trained checkpoint shows the broadest transfer, reaching 78-89% across all eval domains. The Financial checkpoint transfers worst, but no cross-domain pair drops below 70% MCQ accuracy.

CE utilization is similar. The diagonal entries (in-domain) range from 68-77%. Off-diagonal CE utilization is lower (53-79%), with the largest drops when evaluating financial checkpoints on Gutenberg data; evaluating Gutenberg checkpoint on financial data actually yields an improvement. The continuation metric is more sensitive to domain shift than MCQ accuracy, likely because it evaluates the full token distribution rather than just whether specific facts are preserved.

Iterative compaction: the path to unbounded context

A single STILL pass compresses a fixed-length context (e.g., 8K tokens → 1024 latents). But the compactor can be applied iteratively: process the first chunk of a document, compact it, prepend the compact cache to the next chunk, compact the combined cache, and

repeat. After N passes, the final compact cache represents the entire document in a fixed number of latent positions, regardless of how long the original was.

The key question is whether the perceiver can handle its own output as input. That is, compact KV entries from a previous pass occupy a different region of representation space than real KV entries from the LLM. The natural training strategy is to randomize the number of passes per training step (sampling from e.g. $\{1, 2, 4, 8\}$), so the perceiver sees both fresh and previously-compacted input during training.

We are currently evaluating iterative compaction with the identity-init architecture, both with the current setup and training explicitly for re-compaction. Results will follow.

Discussion

The central bet of this work was that KV cache compaction is regular enough across contexts to be amortized into a learned forward pass. The results confirm this: a 71M-parameter perceiver, trained on a few thousand documents from a single domain, generalizes to held-out documents from both the same and different domains with minimal quality loss. The learned compression function captures something about the structure of attention in Qwen3-4B that is not context-specific. We are now in the process of scaling this up to massive compression (256x) and iterative compaction.

Towards continual learning

STILL is a first step on what we see as a longer road to continual learning in LLMs. The current landscape of "model memory" has two extremes and nothing in between:

- **Lossless but expensive.** The KV cache stores every token verbatim. Perfect fidelity, but memory and compute scale linearly with context length. When the window fills up, everything is lost.
- **Compressed but lossy.** Fine-tuning bakes experience into weights, but it's slow, coarse-grained, and prone to catastrophic forgetting. RAG and markdown-style memory files are brittle workarounds that store facts outside the model's native representations.

What's missing is the middle layer, i.e., a compressed working memory that preserves the structure and fidelity of the KV cache at a fraction of the cost. This is what STILL provides: a differentiable, learned compression of the model's own internal representations, stored in a format the model can natively attend to.

But compressed memory alone is only the first of three capabilities we think are needed:

1. *Compressed working memory (this work)*. A learned compactor that produces compact KV caches the LLM can attend to as if they were real context. STILL demonstrates this is feasible: 8x compression with 90%+ factual retention, generalizing across domains, with a forward-pass cost. Applied iteratively, it should extend to unbounded context lengths with fixed memory.
2. *Learned context management*. The model should learn to manage its own memory in deciding when to compress, what compression ratio to use, and what information to prioritize. This means integrating STILL into the model's action space and training with RL so the model can learn context management policies from reward signals. Rather than compaction being a fixed post-processing step, the model would invoke it as a tool: "My context is getting long, compress the older conversation but keep the recent code changes at full fidelity." The perceiver's differentiability makes it compatible with policy gradient methods, and its fixed computational cost makes it practical to invoke repeatedly during generation.
3. *Memory consolidation into weights*. The most speculative piece: can we project what's been learned in compressed KV caches back into the model's weights? One path is a hypernetwork that takes a compressed KV cache, representing a body of experience, and outputs a LoRA adapter for the model. This would be the mechanism for turning short-term compressed memory into durable long-term knowledge, closing the loop between in-context learning and weight updates. If the compactor learns to produce structured, information-dense representations (which the cross-domain transfer results suggest it does), these representations might be a better substrate for weight projection than raw text or token-level gradients.

Together, these three capabilities would produce an LLM that accumulates knowledge across conversations: compressing recent context into working memory, managing that memory actively, and periodically consolidating important patterns into its weights. Not a chatbot that forgets everything at the end of a session, but an agent that gets better at its job over time.

We're early on this road. STILL demonstrates that the first capability (compressed working memory) is achievable with current methods. The next steps are extending to iterative compaction for unbounded contexts, training a universal compactor that works across domains, and beginning the integration with RL for learned context management.

Related work

KV cache compaction. [Attention Matching](#) (Zweiger et al., 2026) provides per-head analytical compaction via key selection, NNLS beta fitting, and least-squares value reconstruction. [Cartridges](#) (Eyuboglu et al., 2025) optimizes compact caches end-to-end via gradient descent. Both operate per-context at inference time. Token eviction methods

(H2O, ScissorHands, StreamingLLM) drop tokens based on attention scores but cannot synthesize new representations.

Perceiver architectures. The Perceiver (Jaegle et al., 2021) introduced learned latent queries for handling long inputs via cross-attention bottlenecks. Perceiver IO (Jaegle et al., 2022) extended this to structured outputs. Perceiver Resampler in Flamingo (Alayrac et al., 2022) applied the same idea to vision-language models. STILL applies the perceiver bottleneck to KV cache compression, a setting where the input (KV cache entries) is highly structured and the output must be compatible with a frozen LLM's attention mechanism.

Knowledge distillation. Our KL distillation objective follows standard practice (Hinton et al., 2015), with the frozen full-cache model as the teacher and the compact-cache model as the student. The key difference from model distillation is that we distill the cache, not the model.

Conclusion

STILL demonstrates that KV cache compaction can be amortized into a learned forward pass. A small perceiver, trained via KL distillation on a frozen LLM, achieves up to 92% extractive accuracy at 8x compression, generalizes across domains, and scales monotonically to 8192 latents. Three architectural fixes (un-rotate/re-rotate RoPE with perceiver-internal positional encoding, removing the final norm, and identity initialization with biased attention projections) were all necessary to unlock this scaling behavior.

But the goal is not only efficient inference. Compressed KV caches are a new kind of model memory: denser than raw context, richer than weight updates, and natively compatible with the model's attention mechanism. We see this as the foundation for continual learning: models that compress experience into working memory, learn to manage that memory actively, and eventually consolidate knowledge into their weights. STILL is the first step, and iterative compaction should extend it beyond any fixed context window.

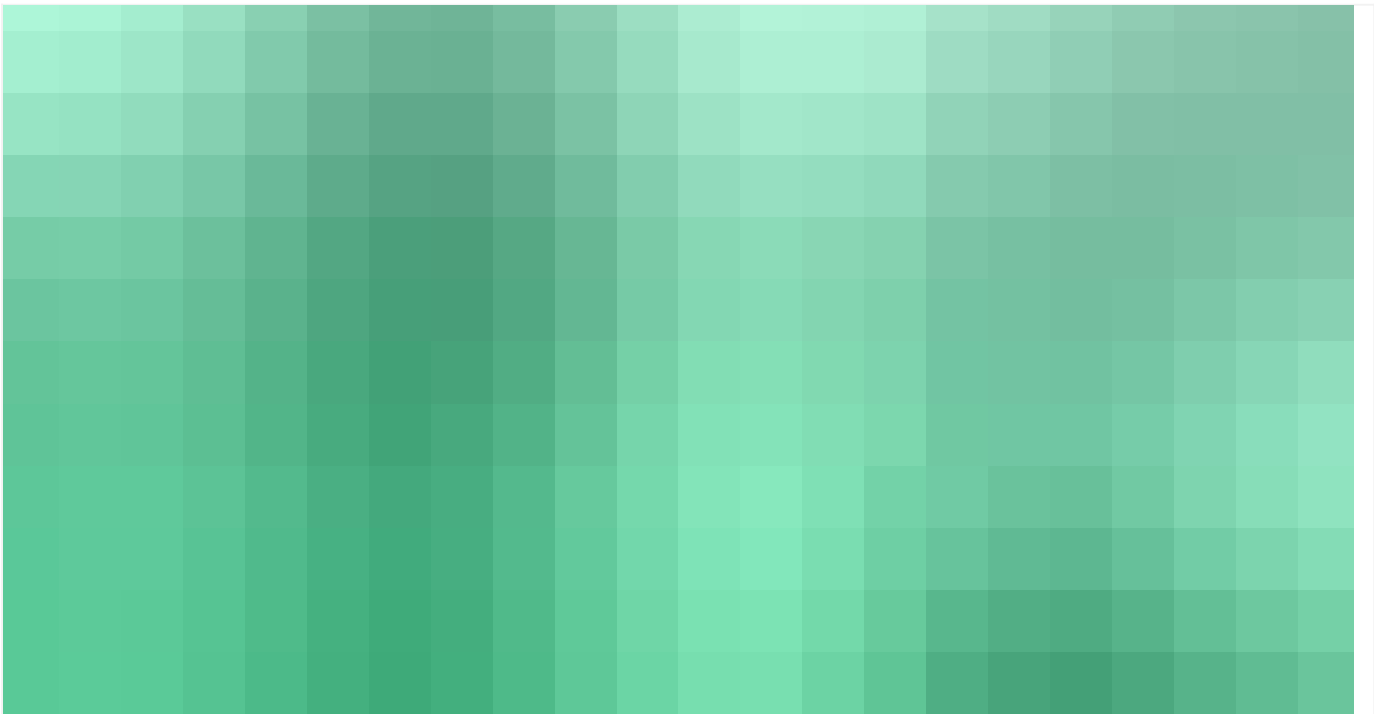
For attribution in academic contexts, please cite this work as:

O'Neill, Charles. "Towards infinite context windows: neural KV cache compaction." Baseten Research, April 1, 2026. <https://www.baseten.co/research/towards-infinite-context-windows-neural-kv-cache-compaction>

BibTeX citation

```
@misc{o'neill2026still,  
  author      = {O'Neill, Charles},  
  title       = {Towards Infinite Context Windows: Neural {KV} Cache  
Compaction},  
  year        = {2026},  
  month       = April,  
  day         = {1},  
  howpublished = {Baseten Research},  
  url         = {https://www.baseten.co/research/towards-infinite-context-  
windows-neural-kv-cache-compaction/},  
}
```

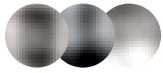
Other research



RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



CHARLES O'NEILL • 2 others



RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.



CHARLES O'NEILL • 4 others

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Продукт

Специализированное логическое устройство

API-интерфейсы моделей

Обучение

Frontier Gateway

Платформа логического вывода Baseten

Исполняемые среды моделей

Инфраструктура

Мультиоблако

Варианты развертывания

- Облачное
- Собственное
- Гибридное

Встраиваемая инженерия

Инженеры , развернутые на переднем крае

Модальности

- Транскрипция
- Генерация изображений
- Преобразование текста в речь
- Большие языковые модели
- Комплексный ИИ
- Эмbedding

ОТРАСЛИ

- Предприятие
- Здравоохранение

Разработчик

- Библиотека моделей
- Документация
- Список изменений

Ресурсы

- Исследования
- Клиенты
- Блог
- Руководства
- Мероприятия
- Партнеры
- Калькулятор сбережений
- Центр доверия
- О нас
- Стартапы
- Вакансии
- Контакты

Популярные модели

- GLM 5.2
- Kimi K2.7 Code
- DeepSeek V4
- GPT OSS 120B
- Whisper Large V3
- NVIDIA Nemotron 3 Ultra
- Посмотреть все

Юридическая информация

- Правила и условия
- Политика конфиденциальности
- Соглашение об уровне обслуживания

● ВСЕ СИСТЕМЫ В НОРМЕ



